

CSE1910 Final Project Report

Reaction Training Dummy: NeuroLight Reflex Trainer

Yenuka Somathilaka

STEM Collegiate, Grade 10

June 11, 2025

Contents

Section 1: Connection to CTS Courses	2
Section 2: Project Proposal and Planning	2
Section 3: Design and Analysis	4
Section 4: Project Development	6
Section 5: Testing, Debugging, and Evaluation	11
Section 6: Full Arduino Source Code (with Comments)	12

Section 1: Connection to CTS Courses

This project connects to **CSE1120: Structured Programming 2** for the core Arduino development and **CSE1210: Client-Side Scripting 1** for making the presentation website. Key outcomes from CSE1120 include designing algorithms using structured programming constructs like selection and iteration, translating these into functional code using appropriate data types and operators, and debugging the resulting program. For CSE1210, the essential outcomes involve designing and writing markup language documents (like HTML) to structure content for the web.

The integration of these skills is central to the project's success. CSE1120 principles guide the development of the Arduino code, ensuring logical flow for reading inputs, processing data, and making outputs using loops and conditions. CSE1210 skills are applied to build the project website using HTML to structure the required content (project description, design documents, code snippets, evaluation) and CSS for visual presentation and styling.

Section 2: Project Proposal and Planning

2.1 Project Proposal

Project Idea:

The project involves designing and building a wireless cognitive reaction training system using ESP32-C3 development boards, ultrasonic sensors, arcade-style buttons, and LED strips. Each station will detect user actions (such as proximity or button press) and communicate wirelessly via ESP-NOW to control lights and measure reaction times.

Purpose:

Reaction training lights are valuable tools for cognitive therapy, especially for older adults and individuals with dementia. By improving reaction speed, hand-eye coordination, and cognitive attention, this project aims to create an engaging, therapeutic system that supports mental sharpness and physical responsiveness. This technology can provide accessible, low-cost therapy to help slow cognitive decline and maintain motor function in aging populations.

Deliverables:

- Fully functional wireless cognitive training system with at least five ESP32 stations.
- ESP32 program implementing ESP-NOW communication between nodes.
- Working integration of ultrasonic sensors, arcade buttons, and LED light feedback.
- HTML and CSS website documenting the project design, code, and testing results.

2.2 Timeline and Milestones

Milestone	Planned Date	Details/Notes

Project Idea Submitted	Week 2	Submit final project idea (Cognitive Reaction Training System).
Proposal Approved	Week 2	Receive instructor approval.
Design Document Completed	Week 3 and 4	Create communication flowcharts, wiring diagrams, and pseudocode.
Hardware Assembly Started	Week 5	Wire sensors, arcade buttons, LEDs to ESP32 boards.
Mid-Project Checkpoint	Week 5	Achieve successful ESP-NOW communication between two nodes.
Testing and Debugging Phase	Week 6	Complete full system testing with multiple nodes.
Website Development Finalized	Week 7	Build and upload project website.
Final Evaluation	Week 8	Evaluate system performance: speed, reliability, usability.
Presentation Ready	Week 8	Prepare and finalize project presentation materials.

2.3 Terminology, Tools, and Resources

Important Terms:

- **ESP-NOW** — A low-power, low-latency, peer-to-peer wireless communication protocol developed by Espressif.
- **Ultrasonic Sensor** — A device that measures distance by using ultrasonic waves.

Tools/Technologies:

Tool/Tech	Purpose in Project
ESP32-C3 Board	Main microcontroller for each station.
HC-SR04 Ultrasonic Sensors (Mini)	Detect user's hand movement or proximity.
Arcade Style Buttons (with LED)	Alternate input method (button press).
LED Light Strips (5V)	Visual feedback and effects.
LiPo Battery + Boost Converter	Portable 5V power supply for each station.
Arduino IDE	Code and program the ESP32 boards.
HTML/CSS	Build and style the project documentation website.

Resources:

- ESP32-C3 development boards (5 units minimum)
- Small ultrasonic sensors

- Arcade buttons with built-in LEDs
- LED light strips (WS2812B preferred)
- LiPo batteries (3.7V 1000-1500mAh) + 5V boost converters
- Breadboards, jumper wires, and 3D printed or laser-cut cases (optional)

2.4 Health, Safety, and Assessment Standards

Health/Safety Concerns:

- Handle LiPo batteries carefully; use protective circuits to avoid overcharging or short circuits.
- Secure all wires and modules to prevent accidental disconnection or electric shorting.
- Ensure the lights and sensor sounds are gentle and not overwhelming for individuals with dementia.

Section 3: Design and Analysis

3.1 Decompose the Project

Host ESP32 Module Flowchart:

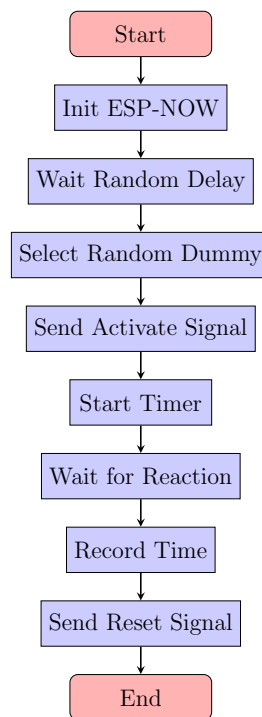


Figure 1: Flowchart for Host ESP32 Module

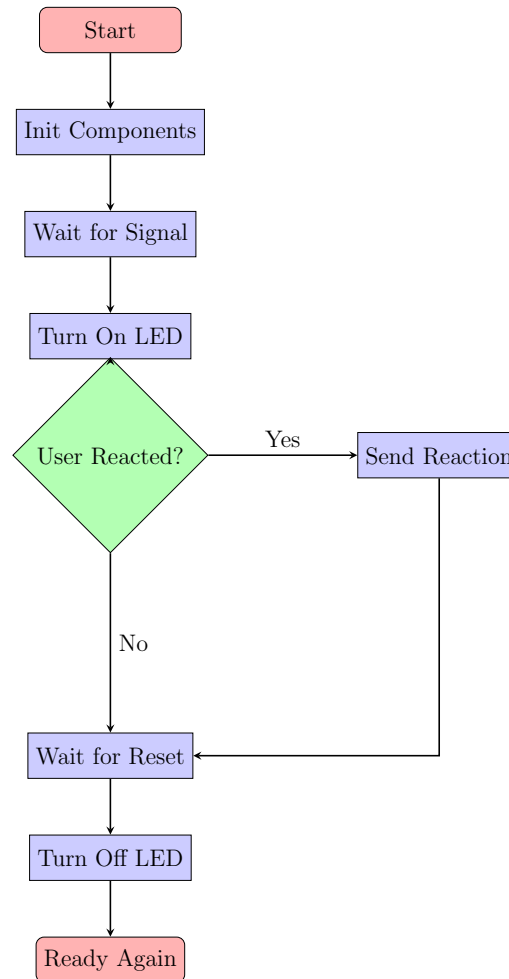
Dummy ESP32 Module Flowchart:

Figure 2: Flowchart for Dummy ESP32 Module

Inputs and Outputs:

- **Inputs:**
 - Activation signal from host
 - Sensor detection or button press by user
- **Outputs:**
 - LED visual cue
 - Reaction time message sent to host

3.2 Website Design

The project includes a clean, responsive website:

- **Home Page** — Project intro and objectives
- **About Me Page** — About me
- **Project Page** — Project report, data, and detailed documentation



Figure 3: Wireframe of the Project Website

3.3 Integration of Skills

This project applies a range of technical skills:

- **Embedded Programming** — Writing modular Arduino code for ESP32
- **Wireless Communication** — Using ESP-NOW for peer-to-peer node updates
- **Sensor Integration** — Detecting proximity and input through hardware
- **Client-side Web Development** — Creating a visually structured HTML/CSS website
- **Project Coordination** — Organizing hardware, coding, testing, and documentation

Section 4: Project Development

4.1 Develop Your Project in Modules

Input Stage

Each client ESP32-C3 is equipped with an ultrasonic sensor (HC-SR04). The sensor detects the user's hand to stop the timer.

Input Code Snippet (Client):

```
// HC-SR04 input
#define TRIG_PIN 2
#define ECHO_PIN 3

long getDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH);
    return duration * 0.034 / 2;
}
```

Each board listens for the signal from the master via ESP-NOW and reacts accordingly.

Processing Stage

The master randomly selects a client board (by MAC address), sends a message to trigger a LED, and starts a timer. The client waits for user detection and responds to the master with the reaction time.

Master MAC Mapping (Example):

```
uint8_t macs[5][6] = {
    {0xD8, 0x3B, 0xDA, 0x27, 0x8D, 0xD4},
    {0xD8, 0x3B, 0xDA, 0x27, 0xAA, 0x10},
    {0xD8, 0x3B, 0xDA, 0x27, 0x92, 0xB4},
    {0x84, 0xF7, 0x03, 0x09, 0xFB, 0x64},
    {0xD8, 0x3B, 0xDA, 0x12, 0x87, 0x64}
};
```

Processing Code Snippet (Master):

```
int selected = random(0, 5);
esp_now_send(macs[selected], (uint8_t *)"ACT", 3);
unsigned long startTime = millis();
```

Processing Code Snippet (Client):

```
void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
    if (memcmp(incomingData, "ACT", 3) == 0) {
        active = true;
        ledOn = random(0, 4);
        digitalWrite(ledPins[ledOn], HIGH);
        timerStart = millis();
    }
}
```

Output Stage

Clients send the reaction time back to the master. Master can log or display it. LEDs give visual cues, and timers measure performance.

Output Code Snippet (Client):

```
if (active) {
  if (getDistance() < 10) {
    active = false;
    digitalWrite(ledPins[ledOn], LOW);
    unsigned long reactTime = millis() - timerStart;
    esp_now_send(masterMAC, (uint8_t*)&reactTime, sizeof(reactTime));
  }
}
```

Output Code Snippet (Master):

```
void OnDataRecv(const uint8_t *mac, const uint8_t *data, int len) {
  unsigned long reaction;
  memcpy(&reaction, data, sizeof(reaction));
  Serial.print("Reaction Time (ms): ");
  Serial.println(reaction);
}
```

4.2 Source Code of Website

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="StylesMainPage.css" />
  <title>Home - Reaction Trainer</title>
</head>
<body>
  <ul class="center-nav">
    <li><a class="active" href="index.html">Home</a></li>
    <li><a href="Project.html">Project</a></li>
    <li><a href="AboutMe.html">About Me</a></li>
  </ul>
  <div class="header">
    <h1>NeuroLight Reflex Trainer</h1>
    <p>Enhancing reaction time using ESP32 and ultrasonic sensing</p>
  </div>
  <div class="description">
    <p>This system uses multiple ESP32 modules and ultrasonic sensors to measure hand reaction speed. LEDs provide visual cues. Ideal for athletes and cognitive training.</p>
  </div>
</body>
</html>
```

Project.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="StylesMainPage.css">
  <title>Project</title>
</head>
<body>
  <ul class="center-nav">
    <li><a href="index.html">Home</a></li>
    <li><a class="active" href="Project.html">Project</a></li>
    <li><a href="AboutMe.html">About Me</a></li>
  </ul>
  <div class="project-card">
    <h2>Project Description</h2>
    <p>The NeuroLight Reflex Trainer is designed to improve reaction times through the use of sensors and LED cues. It is built using ESP32 microcontrollers and ultrasonic distance sensors. The trainer is ideal for both athletes and individuals with cognitive challenges, such as dementia, to support hand-eye coordination and mental alertness.</p>
    <p>This project was developed as part of the CSE1910 course to demonstrate understanding of embedded systems, real-time communication (ESP-NOW), and responsive user interfaces.</p>
  </div>
</body>
</html>
```

AboutMe.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="StylesMainPage.css">
  <title>About Me</title>
</head>
<body>
  <ul class="center-nav">
    <li><a href="index.html">Home</a></li>
    <li><a href="Project.html">Project</a></li>
    <li><a class="active" href="AboutMe.html">About Me</a></li>
  </ul>
  <div class="about-card">
    <h2>About the Creator</h2>
    <p>Hello! My name is Yenuka Somathilaka. Im a Grade 10 student at STEM Collegiate with a passion for robotics, programming, and innovation. This project combines my interests in electronics and problem-solving to create a tool that can help others while demonstrating skills learned in CSE1910.</p>
  </div>
</body>
</html>
```

StylesMainPage.css

```
/* General page styling */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f4f4f4;
}

.center-nav {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: #333;
  display: flex;
  justify-content: center;
}

.center-nav li {
  display: inline;
}

.center-nav li a {
  display: inline-block;
  color: white;
  text-align: center;
  padding: 14px 20px;
  text-decoration: none;
}

.center-nav li a:hover {
  background-color: #111;
}

.center-nav li a.active {
  background-color: #4CAF50;
}

.header {
  text-align: center;
  padding: 50px;
  background-color: #e2e2e2;
}

.description {
  text-align: center;
  margin: 30px auto;
  width: 80%;
  font-size: 18px;
}

.about-card, .project-card {
  background-color: white;
}
```

```
margin: 30px auto;
padding: 20px;
width: 80%;
border-radius: 12px;
box-shadow: 0 4px 8px rgba(0,0,0,0.1);
}

.center-button {
  text-align: center;
  margin-top: 20px;
}

.center-button button {
  padding: 10px 20px;
  font-size: 16px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.center-button button:hover {
  background-color: #45a049;
}
```

Section 5: Testing, Debugging, and Evaluation

During the development of the Reaction Training Dummy project, thorough testing and debugging were essential to ensure reliable and responsive operation.

Testing:

Each client board was individually tested to verify the ultrasonic sensor accurately detected hand presence within the specified range (less than 10 cm). The LEDs were tested to confirm they lit correctly upon activation and turned off when the sensor was triggered. The wireless communication using ESP-NOW was tested to ensure messages between the master and clients were sent and received without delay or loss.

Debugging:

Initial challenges included conflicting declarations in the code and timing issues causing LED activation delays. These were resolved by carefully reviewing variable scopes, improving synchronization between the master and clients, and ensuring consistent intervals between activations. Adjustments to the sensor reading logic helped reduce false positives and missed detections.

Evaluation:

The system successfully met the project requirements by activating one client at a time and responding promptly to user input. The consistent 1000 ms activation interval provided a steady training pace. Using ESP-NOW allowed for low-latency communication, making the

reflex training responsive and effective.

Some limitations were noted, such as occasional sensor noise and the fixed LED activation pattern lighting all three LEDs simultaneously on a client, which could be improved in future iterations for more varied training stimuli.

Overall, the project demonstrated a functional reflex training system suitable for athletes or users needing to improve reaction time, with room for enhancements in sensor accuracy and user interaction complexity.

Section 6: Full Arduino Source Code (with Comments)

6.1 Master ESP32 Code

```
// Include required libraries
#include <esp_now.h>
#include <WiFi.h>

// List of MAC addresses for each client (dummy) board
uint8_t clients[][6] = {
  {0xD8, 0x3B, 0xDA, 0x27, 0x8D, 0xD4}, // Board 1
  {0xD8, 0x3B, 0xDA, 0x27, 0xAA, 0x10}, // Board 2
  {0x84, 0xF7, 0x03, 0x09, 0xFB, 0x64}  // Board 3
};

int currentIndex = -1; // Index of the last activated client
bool waiting = false; // True if waiting for a response from client
unsigned long lastSend = 0; // Time when the last signal was sent
const int maxWait = 700; // Max wait time (ms) before skipping if no hand detected

// Callback for receiving data from clients
void onDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  if (len == 1 && data[0] == 0) {
    waiting = false; // Reset waiting flag if client signals done
  }
}

// Send an activation signal to a specific client
void sendToClient(int index, uint8_t value) {
  esp_now_send(clients[index], &value, sizeof(value));
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA); // Set WiFi to station mode
  WiFi.disconnect();

  // Initialize ESP-NOW
  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed");
    return;
  }
}
```

```

// Register callback for receiving data
esp_now_register_rcv_cb(onDataRecv);

// Register each client as a peer
for (int i = 0; i < 3; i++) {
    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, clients[i], 6);
    peer.channel = 0;
    peer.encrypt = false;

    if (!esp_now_is_peer_exist(clients[i])) {
        esp_now_add_peer(&peer);
    }
}

void loop() {
    unsigned long now = millis();

    if (!waiting) {
        // Pick a new client different from the last one
        int nextIndex;
        do {
            nextIndex = random(0, 3);
        } while (nextIndex == currentIndex);

        currentIndex = nextIndex;
        sendToClient(currentIndex, 1); // Activate the selected client
        waiting = true;
        lastSend = now;
    }

    // If hand not detected in time, move on to next
    if (waiting && (millis() - lastSend > maxWait)) {
        waiting = false;
    }
}

```

6.2 Dummy (Client) ESP32 Code

```

// Include required libraries
#include <esp_now.h>
#include <WiFi.h>

// Pin definitions for ultrasonic sensor and LEDs
#define TRIG_PIN 2
#define ECHO_PIN 3
#define LED1_PIN 4
#define LED2_PIN 5
#define LED3_PIN 6

uint8_t active = 0; // Set to 1 when this board should activate LEDs

```

```
// Callback for receiving data from master
void onDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    if (len == 1) {
        active = data[0]; // Activate if value is 1, deactivate if 0
    }
}

void setup() {
    Serial.begin(115200);

    // Set up pins for sensor and LEDs
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(LED1_PIN, OUTPUT);
    pinMode(LED2_PIN, OUTPUT);
    pinMode(LED3_PIN, OUTPUT);

    WiFi.mode(WIFI_STA); // Set WiFi to station mode
    WiFi.disconnect();

    // Initialize ESP-NOW
    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW init failed");
        return;
    }

    // Register callback for receiving data
    esp_now_register_recv_cb(onDataRecv);
}

// Measure distance using HC-SR04 ultrasonic sensor
float getDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // Timeout 30ms
    return duration * 0.034 / 2; // Convert to centimeters
}

void loop() {
    if (active) {
        // Turn on all LEDs to signal activation
        digitalWrite(LED1_PIN, HIGH);
        digitalWrite(LED2_PIN, HIGH);
        digitalWrite(LED3_PIN, HIGH);

        float distance = getDistanceCM();
        // If hand is detected (distance < 10cm), turn off LEDs and signal done
        if (distance > 0 && distance < 10) {
            active = 0;
        }
    }
}
```

```
digitalWrite(LED1_PIN, LOW);
digitalWrite(LED2_PIN, LOW);
digitalWrite(LED3_PIN, LOW);

// Send '0' back to master to indicate completion
uint8_t done = 0;
esp_now_send(info->src_addr, &done, 1);
}
}
}
```